

CSS I — Fundamentos, variables, funciones, colores y tipografía

En esta sesión

En esta sesión vas a construir una base más realista de CSS para trabajar en proyectos actuales.

El objetivo no es memorizar propiedades sueltas, sino comprender:

- cómo funciona CSS;
- cómo se organizan las reglas;
- cómo se construyen estilos reutilizables;
- cómo usar variables y funciones útiles;
- cómo trabajar colores y tipografía con criterio;
- y cómo preparar una base sólida para Flexbox, Grid, responsive y componentes.

1. Introducción

En HTML has trabajado la estructura del documento y la semántica. CSS entra cuando esa estructura necesita presentación visual: espaciado, color, tipografía, tamaño, jerarquía y composición.

Idea clave

HTML responde a la pregunta **qué es cada parte del contenido**.

CSS responde a la pregunta **cómo se presenta ese contenido**.

Aunque más adelante uses librerías, componentes o herramientas como Tailwind, CSS sigue siendo la base de todo lo que ves en una interfaz web. Si no entiendes CSS, es muy fácil terminar copiando estilos sin saber por qué funcionan o por qué dejan de funcionar.

Trabajar bien CSS te ayuda a:

Beneficio	Qué aporta en la práctica
Control visual real	Puedes modificar interfaces con criterio
Mantenimiento	Entiendes qué regla afecta a cada parte
Escalabilidad	Evitas que el CSS se vuelva caótico
Transición a herramientas posteriores	Comprendes mejor React, Tailwind o sistemas de diseño
Depuración	Detectas antes problemas de cascada, jerarquía o herencia

En desarrollo web sigue siendo útil pensar en tres capas:

Capa	Función principal	Tecnología habitual
Estructura	Define qué representa cada parte	HTML
Presentación	Define cómo se ve	CSS
Comportamiento	Define cómo responde la interfaz	JavaScript

En esta sesión vas a trabajar la base del lenguaje, pero con una ampliación importante respecto a una introducción mínima: además de sintaxis, selectores, cascada, colores y tipografía, vas a ver **variables CSS**, **funciones CSS útiles** y una primera aproximación práctica a la **carga de fuentes**.

Referencias del apartado 1

- MDN Web Docs. *CSS: Cascading Style Sheets*.
- MDN Web Docs. *How CSS works*.
- MDN Web Docs. *Getting started with CSS*.

2. Cómo funciona CSS

Para trabajar bien con CSS no basta con saber que “sirve para poner estilos”. Necesitas entender cómo se incorpora al documento, cómo se escribe una regla y cómo decide el navegador qué estilos aplicar.

Formas de aplicar CSS

Existen tres formas habituales de aplicar CSS:

Forma	Ejemplo	Uso recomendado
Hoja externa	<code><link rel="stylesheet" href="./styles.css"></code>	Opción habitual en proyectos reales
Etiqueta <code><style></code>	CSS dentro del <code>head</code>	Pruebas o ejemplos rápidos
Inline	<code>style="color: steelblue;"</code>	Casos muy concretos, no como estrategia normal

La hoja externa es la forma de trabajo habitual porque separa estructura y presentación, mejora el mantenimiento y permite reutilizar estilos.

```
<head>
  <meta charset="UTF-8">
  <title>Mi página</title>
  <link rel="stylesheet" href="./styles.css">
</head>
```

Forma habitual de trabajo

En un proyecto real, lo normal es trabajar con una hoja de estilos externa o con varias hojas organizadas por bloques.

Sintaxis básica de una regla

CSS funciona a partir de reglas.

```
selector {
  propiedad: valor;
}
```

Ejemplo:

```
h1 {
  color: steelblue;
```

```
}
```

Partes de una regla CSS:

Parte	Función
Selector	Indica a qué elementos se aplica la regla
Propiedad	Indica qué aspecto se modifica
Valor	Indica con qué configuración concreta se modifica
Declaración	Conjunto propiedad + valor
Bloque de reglas	Conjunto completo entre llaves

Ejemplo con varias declaraciones:

```
p {  
  color: #333333;  
  font-size: 16px;  
  line-height: 1.5;  
}
```

Errores frecuentes:

- olvidar los dos puntos;
- olvidar el punto y coma;
- cerrar mal las llaves;
- escribir mal propiedades;
- usar valores no válidos.

Selectores básicos

Los selectores determinan qué elementos del HTML reciben una regla.

Selector	Ejemplo	Uso típico
Etiqueta	<code>p { ... }</code>	Estilos base generales
Clase	<code>.card { ... }</code>	Componentes o bloques reutilizables
ID	<code>#main-header { ... }</code>	Casos puntuales y únicos

Selector	Ejemplo	Uso típico
Agrupado	<code>h1, h2, h3 { ... }</code>	Aplicar la misma regla a varios elementos
Universal	<code>* { ... }</code>	Reglas globales muy concretas
Descendiente	<code>nav a { ... }</code>	Elementos dentro de un contexto

Ejemplos:

```
h1 {
  color: steelblue;
}

.card {
  background-color: #f5f5f5;
}

#main-header {
  background-color: #111827;
}

h1,
h2,
h3 {
  font-family: Arial, sans-serif;
}

nav a {
  text-decoration: none;
}
```

Regla práctica

En CSS de proyecto, la clase suele ser el selector más útil y más mantenible.

Cascada, especificidad y herencia

CSS no aplica reglas al azar. Cuando varias afectan al mismo elemento, entran en juego estas tres ideas.

La **cascada** decide qué regla termina aplicándose cuando dos reglas tienen el mismo peso.

```
p {  
  color: black;  
}  
  
p {  
  color: blue;  
}
```

Aquí el texto se verá azul porque la segunda regla aparece después.

La **especificidad** decide qué selector pesa más.

Tipo de selector	Peso relativo
Etiqueta	Bajo
Clase	Medio
ID	Alto

Ejemplo:

```
p {  
  color: black;  
}  
  
.text-highlight {  
  color: orange;  
}  
  
#main-text {  
  color: green;  
}
```

Y en HTML:

```
<p id="main-text" class="text-highlight">Texto destacado</p>
```

Ganará el selector por `id`.

La **herencia** hace que algunas propiedades pasen de un elemento padre a sus hijos, especialmente las de texto.

```
body {  
  font-family: Arial, sans-serif;  
  color: #222222;  
}
```

Propiedades que suelen heredarse:

- `color`
- `font-family`
- `font-size` en ciertos contextos
- `line-height`

Propiedades que no suelen heredarse:

- `margin`
- `padding`
- `border`
- `background-color`

También existe `!important`, pero no debe convertirse en una muleta habitual.

Recomendación

Si sientes que necesitas `!important` constantemente, el problema suele estar en la organización del CSS, no en la falta de fuerza de una regla.

Referencias del apartado 2

- MDN Web Docs. *How to apply CSS to HTML*.
- MDN Web Docs. *CSS selectors*.
- MDN Web Docs. *Cascade*.
- MDN Web Docs. *Specificity*.
- MDN Web Docs. *Inheritance*.

3. Unidades, variables y funciones CSS

Aquí empiezas a entrar en un CSS más actual. No basta con saber poner colores o tamaños; también necesitas crear estilos reutilizables y flexibles.

Unidades más útiles

No todas las unidades sirven para lo mismo.

Unidad	Uso frecuente recomendado
px	Medidas concretas y controladas
%	Anchos relativos
rem	Tipografía y escalas coherentes
em	Medidas relativas al contexto inmediato
vh / vw	Elementos ligados al viewport

Ejemplos:

```
h1 {  
  font-size: 32px;  
}  
  
.container {  
  width: 80%;  
}  
  
p {  
  font-size: 1rem;  
}  
  
.small-note {  
  font-size: 0.875em;  
}  
  
.hero {  
  min-height: 100vh;  
}
```

Regla práctica

En una base de proyecto sencilla, `px`, `rem`, `%` y algo de `vh / vw` cubren una gran parte de necesidades reales.

Variables CSS

Las variables CSS también se conocen como **custom properties**. Permiten guardar valores reutilizables y mantener consistencia.

Se definen normalmente en `:root` cuando quieres que estén disponibles globalmente.

```
:root {  
  --color-primary: #2563eb;  
  --color-text: #1f2937;  
  --space-md: 16px;  
  --radius-md: 12px;  
}
```

Después se usan con `var()`:

```
.button {  
  background-color: var(--color-primary);  
  border-radius: var(--radius-md);  
}  
  
.card {  
  padding: var(--space-md);  
  color: var(--color-text);  
}
```

Esto resulta muy útil para:

- colores;
- espaciado;
- radios;
- tipografía;
- sombras;
- anchos máximos.

Idea clave

Las variables CSS no son un adorno “avanzado”. Son una forma muy práctica de evitar repetición y crear consistencia visual.

También puedes definir variables a nivel de componente:

```
.alert {  
  --alert-bg: #fee2e2;  
  --alert-text: #991b1b;  
  
  background-color: var(--alert-bg);  
  color: var(--alert-text);  
}
```

Funciones CSS útiles

Hay varias funciones que resultan especialmente útiles en el día a día.

calc()

Permite hacer cálculos sencillos.

```
.main {  
  width: calc(100% - 32px);  
}
```

min()

Elige el valor más pequeño.

```
.container {  
  width: min(100%, 1100px);  
}
```

max()

Elige el valor más grande.

```
.card {  
  padding: max(12px, 2vw);  
}
```

clamp()

Define un valor flexible entre un mínimo y un máximo. Es muy útil en tipografía fluida.

```
h1 {  
  font-size: clamp(2rem, 5vw, 4rem);  
}
```

`clamp(min, preferido, max)` significa:

- nunca menos del mínimo;
- intenta usar el valor intermedio;
- nunca más del máximo.

Uso muy práctico

`clamp()` es una de las funciones más útiles para tamaños de texto y espaciados que deben adaptarse sin volverse extremos.

Referencias del apartado 3

- MDN Web Docs. *Values and units*.
- MDN Web Docs. *CSS custom properties*.
- MDN Web Docs. *Using CSS custom properties*.
- MDN Web Docs. *calc()*.
- MDN Web Docs. *min()*.
- MDN Web Docs. *max()*.
- MDN Web Docs. *clamp()*.

4. Colores, fondos y tipografía

Una gran parte del CSS inicial de una interfaz tiene que ver con colores y texto. Aquí no se busca “decorar”, sino construir una base legible, clara y consistente.

Colores en CSS

Las propiedades más frecuentes son:

- `color` para el texto;
- `background-color` para el fondo.

```
p {  
  color: #222222;  
}  
  
section {
```

```
background-color: #f3f4f6;
}
```

Formatos de color más usados:

Formato	Ejemplo	Cuándo resulta útil
Nombre	steelblue	Ejemplos rápidos
Hex	#2563eb	Muy habitual en proyectos
rgb()	rgb(37, 99, 235)	Precisión por canales
hsl()	hsl(217, 91%, 60%)	Ajuste intuitivo de tono y saturación

Ejemplos:

```
h1 {
  color: steelblue;
}

button {
  background-color: rgb(37, 99, 235);
}

.badge {
  background-color: hsl(217, 91%, 60%);
}
```

Buena práctica

Cuando el CSS afecta a texto, la legibilidad importa más que la estética superficial.

Fondos y gradientes

Además del color plano, puedes usar gradientes para aportar profundidad o jerarquía visual.

```
.hero {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
```

Los gradientes lineales pueden ser útiles en:

- cabeceras;
- bloques destacados;
- botones;
- banners;
- fondos decorativos sutiles.

Conviene usarlos con medida. No todo necesita gradientes.

Tipografía y texto

Las propiedades tipográficas más frecuentes son:

Propiedad	Función
<code>font-family</code>	Define la familia tipográfica
<code>font-size</code>	Define el tamaño del texto
<code>font-weight</code>	Define el grosor
<code>font-style</code>	Permite cursiva u otras variantes
<code>line-height</code>	Controla la altura de línea
<code>text-align</code>	Alinea el texto
<code>text-decoration</code>	Añade o quita decoraciones
<code>text-transform</code>	Transforma visualmente el texto
<code>letter-spacing</code>	Ajusta el espacio entre letras

Ejemplo:

```
body {  
  font-family: Arial, sans-serif;  
  color: #1f2937;  
}  
  
h1 {  
  font-size: 2rem;  
  font-weight: 700;  
  text-align: center;  
}  
  
p {
```

```
font-size: 1rem;
line-height: 1.6;
}

a {
  text-decoration: none;
}
```

Regla práctica

Un buen `line-height` mejora mucho la lectura, especialmente en párrafos y descripciones.

Google Fonts

Cargar una fuente externa es una necesidad frecuente en proyectos reales. Una opción muy común es usar Google Fonts.

Puedes incorporarla con un enlace en el `head` y después aplicarla en CSS.

Ejemplo de idea general:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;600;700&display=swap" rel="stylesheet">
```

Y después:

```
body {
  font-family: "Inter", Arial, sans-serif;
}
```

Aquí conviene entender dos ideas:

- siempre debes definir una familia de respaldo;
- no necesitas cargar muchas fuentes distintas para que una interfaz funcione bien.

Error frecuente

Usar demasiadas familias tipográficas o demasiados pesos complica la interfaz y empeora la coherencia visual.

Referencias del apartado 4

- MDN Web Docs. *CSS colors*.
- MDN Web Docs. *background*.
- MDN Web Docs. *linear-gradient()*.
- MDN Web Docs. *font-family*.
- MDN Web Docs. *font-size*.
- MDN Web Docs. *line-height*.
- Google Fonts.

5. Primeras decisiones de estilo con criterio

En esta fase no se trata de “probar propiedades hasta que algo quede bien”. Se trata de empezar a decidir con lógica.

Algunas decisiones razonables al principio de un proyecto pueden ser estas:

- definir en `:root` los colores y espaciados principales;
- elegir una familia tipográfica principal y una de respaldo;
- usar `rem` en tamaños de texto y `%` o `min()` en anchuras;
- reservar `px` para detalles muy concretos;
- usar `clamp()` cuando quieras evitar saltos extremos entre móvil y escritorio;
- evitar colores arbitrarios repetidos por todo el CSS;
- empezar a construir una base reutilizable antes de entrar en maquetaciones más complejas.

Ejemplo integrador:

```
:root {
  --font-base: "Inter", Arial, sans-serif;
  --color-text: #1f2937;
  --color-primary: #2563eb;
  --color-surface: #ffffff;
  --space-md: 16px;
  --radius-md: 12px;
```

```
}

body {
  font-family: var(--font-base);
  color: var(--color-text);
  background-color: #f9fafb;
}

.container {
  width: min(100%, 1100px);
  margin-inline: auto;
  padding: var(--space-md);
}

.card {
  background-color: var(--color-surface);
  border-radius: var(--radius-md);
  padding: var(--space-md);
}

h1 {
  font-size: clamp(2rem, 4vw, 3rem);
}
```

Idea clave

Una buena base de CSS no depende de tener muchas propiedades, sino de que las decisiones iniciales sean coherentes y reutilizables.

6. Buenas prácticas iniciales

El CSS se vuelve caótico muy rápido si no desarrollas buenos hábitos desde el principio.

Las primeras buenas prácticas que debes adoptar son estas:

- priorizar clases frente a ids;
- usar nombres claros;
- evitar duplicación innecesaria;
- mantener una propiedad por línea;
- trabajar con orden;
- reutilizar valores mediante variables;

- no abusar de `!important`;
- pensar en consistencia antes que en excepciones.

Tabla de errores frecuentes de inicio:

Error	Problema	Enfoque correcto
Usar <code>id</code> para todo	Aumenta especificidad y reduce flexibilidad	Preferir clases
Repetir colores y tamaños en muchas reglas	Dificulta mantenimiento	Usar variables CSS
Cargar demasiadas fuentes	Empeora coherencia y rendimiento	Elegir pocas y con criterio
Usar <code>text</code> o medidas sin pensar la escala	Rompe consistencia visual	Trabajar una base tipográfica razonable
Probar propiedades sin entender la cascada	Genera confusión constante	Revisar especificidad y herencia

Buena práctica

Nombra pensando en el papel del elemento, no solo en su apariencia puntual. Una clase como `.card-title` suele ser mejor que `.blue-big-text`.

Referencias del apartado 6

- MDN Web Docs. *Writing reusable CSS*.
- MDN Web Docs. *Best practices for styling*.

7. Análisis técnico avanzado y cierre

Muchas personas empiezan CSS memorizando propiedades sueltas. Eso puede servir para copiar ejemplos, pero no para construir interfaces de forma fiable.

Lo verdaderamente importante en esta sesión es comprender que CSS es un sistema:

- las reglas compiten;
- los selectores importan;

- la herencia influye;
- las unidades no son intercambiables;
- las variables permiten consistencia;
- las funciones permiten flexibilidad;
- y la tipografía no es un detalle superficial.

También conviene entender que esta sesión prepara mucho de lo que vendrá después:

- Flexbox y Grid necesitarán una base clara de medidas, spacing y cajas;
- responsive design se apoyará en unidades y funciones como `clamp()` ;
- componentes y metodología CSS funcionarán mejor si ya piensas en reutilización;
- y herramientas posteriores como Tailwind tendrán más sentido si ya entiendes colores, spacing, tipografía y diseño de tokens.

Qué debes retener de esta sesión

- CSS define la presentación visual del contenido.
- La hoja externa es la forma habitual de trabajo.
- Una regla CSS se compone de selector, propiedad y valor.
- La cascada, la especificidad y la herencia explican gran parte del comportamiento del CSS.
- Las unidades deben elegirse con criterio según contexto.
- Las variables CSS ayudan a crear consistencia y mantenimiento.
- `calc()` , `min()` , `max()` y `clamp()` son funciones muy útiles en proyectos reales.
- Colores, fondos y tipografía forman la base visual de una interfaz.
- Google Fonts permite incorporar tipografías externas, pero debe usarse con criterio.
- Una buena base de CSS es pequeña, clara y reutilizable.

Referencias documentales generales

1. MDN Web Docs. *CSS: Cascading Style Sheets*.
2. MDN Web Docs. *How CSS works*.
3. MDN Web Docs. *CSS selectors*.
4. MDN Web Docs. *Cascade*.
5. MDN Web Docs. *Specificity*.
6. MDN Web Docs. *Values and units*.
7. MDN Web Docs. *CSS custom properties*.
8. MDN Web Docs. *calc()*.

9. MDN Web Docs. *clamp()*.
10. MDN Web Docs. *CSS colors*.
11. MDN Web Docs. *background*.
12. MDN Web Docs. *font-family*.
13. Google Fonts.

Mini glosario

Término	Definición breve
Cascada	Mecanismo que decide qué regla se aplica
Especificidad	Peso relativo de un selector frente a otros
Herencia	Transmisión de ciertas propiedades desde un elemento padre
Custom property	Variable CSS definida por el desarrollador
<code>var()</code>	Función para leer una variable CSS
<code>calc()</code>	Función para hacer cálculos sencillos
<code>clamp()</code>	Función para limitar un valor flexible entre mínimo y máximo
Fallback font	Fuente de respaldo si la principal no está disponible
Gradiente	Transición progresiva entre colores
Token de diseño	Valor reutilizable de color, espacio, tipografía u otro recurso visual

Fragmento de ejemplo integrador

HTML

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Curso de HTML y CSS</title>

    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;600;700&display=swap" rel="stylesheet">
```

```

    <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <header class="site-header">
    <div class="container">
      <h1 class="site-title">Curso de HTML y CSS</h1>
      <p class="site-subtitle">
        Aprende a construir interfaces limpias, reutilizables y mantenibles.
      </p>
    </div>
  </header>

  <main class="container">
    <section class="intro-card">
      <h2>Qué vas a aprender</h2>
      <p>
        Empezarás por la estructura HTML y continuarás con estilos,
maquetación,
        responsive design y componentes.
      </p>
    </section>
  </main>
</body>
</html>

```

CSS

```

:root {
  --font-base: "Inter", Arial, sans-serif;
  --color-text: #1f2937;
  --color-surface: #ffffff;
  --color-primary: #2563eb;
  --color-bg: #f8fafc;
  --space-md: 16px;
  --space-lg: 24px;
  --radius-md: 12px;
}

body {
  font-family: var(--font-base);
  color: var(--color-text);
  background-color: var(--color-bg);
}

.container {

```

```
width: min(100%, 1100px);
margin-inline: auto;
padding: var(--space-md);
}

.site-header {
  background: linear-gradient(135deg, #dbeafe, #ede9fe);
}

.site-title {
  font-size: clamp(2rem, 5vw, 3.5rem);
  color: var(--color-primary);
}

.site-subtitle {
  font-size: 1rem;
  line-height: 1.6;
}

.intro-card {
  background-color: var(--color-surface);
  padding: var(--space-lg);
  border-radius: var(--radius-md);
}
```